IT404041 REAL TIME PROGRAMMING · ASSIGNMENT 2 · SMART GREENHOUSE CONTROLLER

CLI Guide

What the screen shows and what happens inside

Generated 2026-11-05

This guide explains, in simple words:

1. why lines keep repeating even when nothing happens;
2. what every kind of line means;
3. step by step, what the system does when an input arrives (with real CLI output).

---

1. The big idea: the system is a loop that never stops

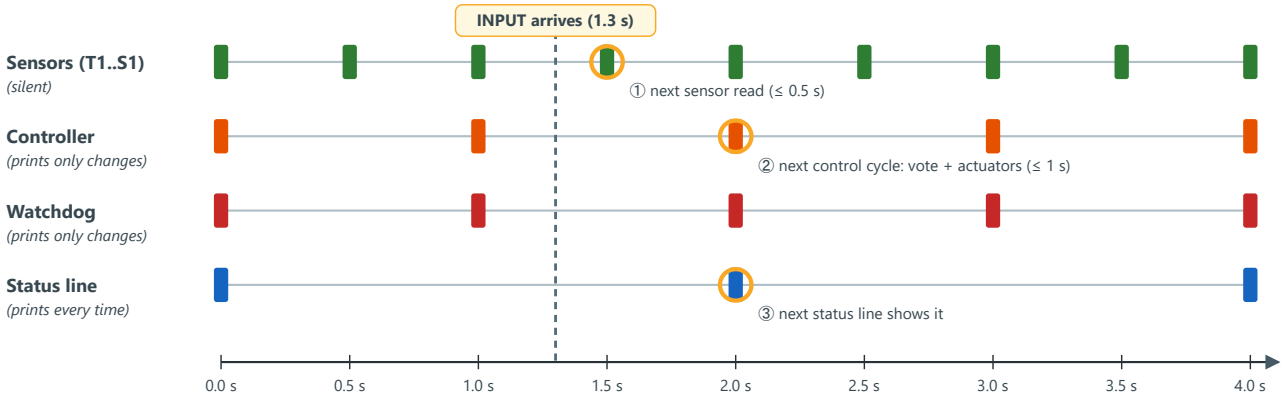
A real-time controller is **always running**, like a heart beating. Even when you send nothing, these jobs run again and again on their own:

Job	How often	What it does	Do you see a line?
5 sensors (T1, T2, T3, H1, S1)	every 0.5 s	read the greenhouse, put the reading in the buffer	no – it would be 10 lines per second
Consumer thread	for every reading	take the reading out, check it, store it	no, only if a reading is rejected
Controller	every 1 s	vote the temperature, switch heater / fan / pump / alarm	only when something **changes**
Watchdog	every 1 s	check every sensor is still sending	only when a sensor **dies or comes back**
Status display	every 2 s	print one summary line	**yes –** `[STATUS] (every 2 s)`
Timing report	every 60 s	print the execution-time table	**yes –** `[TIMING]`

So the lines that "go like a loop" are just `[STATUS]` **every 2 seconds** and `[TIMING]` **every 60 seconds**. They mean "I am alive and this is my current state". They are **not** errors and **not** reactions.

Everything else only appears when something happens.

The system is a loop that never stops – and an input in the middle



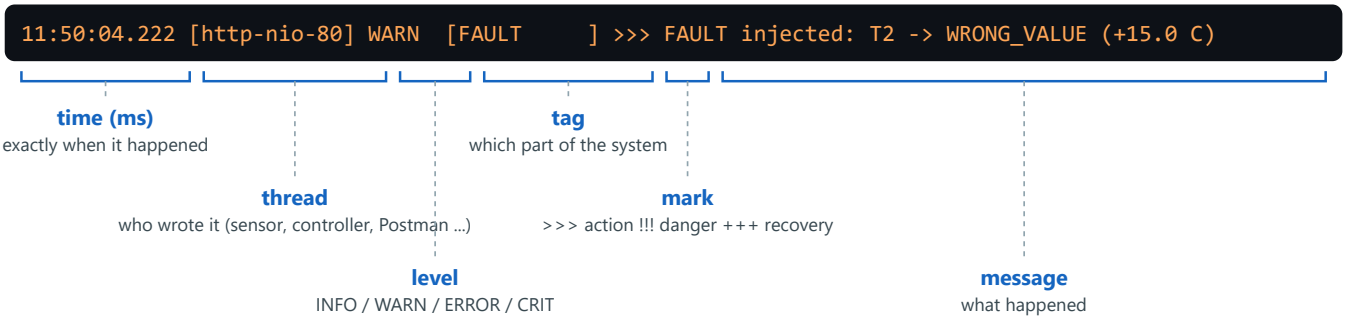
Reaction time: at most 0.5 s (sensor) + 1 s (control cycle). The alarm uses Observe & React, so it does not wait for the control cycle.

---

2. The kinds of lines

Every normal line has the same shape

Anatomy of one CLI line



Part	Meaning
time	Hours:minutes:seconds.**milliseconds** – shows *exactly* when it happened
thread	Which part of the program wrote the line: `sensor-T1`, `consumer`, `controller`, `watchdog`, `display`, `http-nio-80…` (= a Postman request), `fault-menu`
level	`INFO` normal · `WARN` something is not normal · `ERROR` a fault was detected · `CRIT` needs a human
tag	Which part of the system it is about (table below)
mark	`>>>` fault or action · `!!!` safe state / alarm / error · `+++` recovery, back to normal

Tags

Tag	Means	Example
`STATUS`	the repeating line every 2 s – the current state	`(every 2 s) T=25.0C [MASKED 2/3] ... T2:WRONG_VALUE`
`CHECK`	Postman reads the status to test the result – **changes nothing**	`Postman reads the status (test step) -> T=25.1 C, vote MASKED`
`SIMULATOR`	the greenhouse values were changed	`Environment set: T=25.0 C ...` / `HEAT WAVE +14.0 C`
`FAULT`	a sensor was broken or repaired	`FAULT injected: T3 -> DEAD`
`VALIDATION`	an impossible reading was thrown away	`Reading of T2 = 115.0 C REJECTED`
`VOTER`	the voting result **changed**	`OK -> MASKED : T2=39.9 C out-voted`
`WATCHDOG`	a sensor stopped sending / sends again	`T3 FAILED - no heartbeat for 2001 ms`
`RT2`, `RT3`	a hard deadline was measured	`[RT3 HARD] T3 detected dead 1611 ms ... MET`
`OBSERVE`	immediate alarm reaction (no waiting)	`Observe & React: voted T=39.0 C > 35 C`
`ACTUATOR`	heater / fan / pump / alarm switched	`ALARM -> ON (T=39.0 C > 35.0 C)`
`SAFE-STATE`, `ALARM`	safe state entered / active / left	`ENTER SAFE STATE ...` / `EXIT SAFE STATE ...`
`CONTROL`	warning from the controller	`soil sensor S1 not trustworthy - pump OFF`
`DEADLINE`	a task was late (should not happen)	`DEADLINE MISS: ...`

Tag	Means	Example
`TIMING`	the timing table (every 60 s and at the end)	`Total U = 0.014 ... SCHEDULABLE`
`RACE-DEMO`	F4 results	`Unsafe counter = 130173 ... LOST 69827 updates`

The boxes

New input (something arrived):

Smart Greenhouse Controller – CLI (F1: T2 wrong value)

```
================================== NEW INPUT RECEIVED ==================================
  11:50:04.221  from Postman:  POST /api/sensors/T2/fault  {type=WRONG_VALUE, value=15.0}
  WHAT IT MEANS : temperature sensor T2 will report a WRONG value (+15.0 added to the real value)
  EXPECTED      : voter out-votes T2 -> MASKED, no wrong action
  SYSTEM REACTION (lines below):
----------------------------------------------------------------------------------------
11:50:04.222 WARN  [FAULT     ] >>> FAULT injected: T2 -> WRONG_VALUE (+15.0 C)
11:50:04.224 WARN  [VOTER     ] >>> OK -> MASKED : T2=39.9 C out-voted, T1 and T3 agree -> T=25.0 C
11:50:05.225 INFO  [STATUS    ] (every 2 s) T=25.0C [MASKED 2/3] Heater off Fan off Alarm off | T2:WRONG_VALUE
11:50:06.313 INFO  [CHECK     ] Postman reads the status (test step) -> vote MASKED, safe state no

================================== NEW INPUT RECEIVED ==================================
  11:50:06.408  from Postman:  DELETE /api/sensors/T2/fault
  WHAT IT MEANS : temperature sensor T2 is repaired
----------------------------------------------------------------------------------------
11:50:06.408 INFO  [FAULT     ] +++ T2 repaired - sensor works normally again
11:50:07.225 INFO  [VOTER     ] +++ MASKED -> OK : all 3 agree (T1=25.2, T2=24.9, T3=25.1)
```

→ **read the lines under the box** – that is how the system reacted to this input.

Rejected input (invalid data):

Smart Greenhouse Controller – CLI (invalid input)

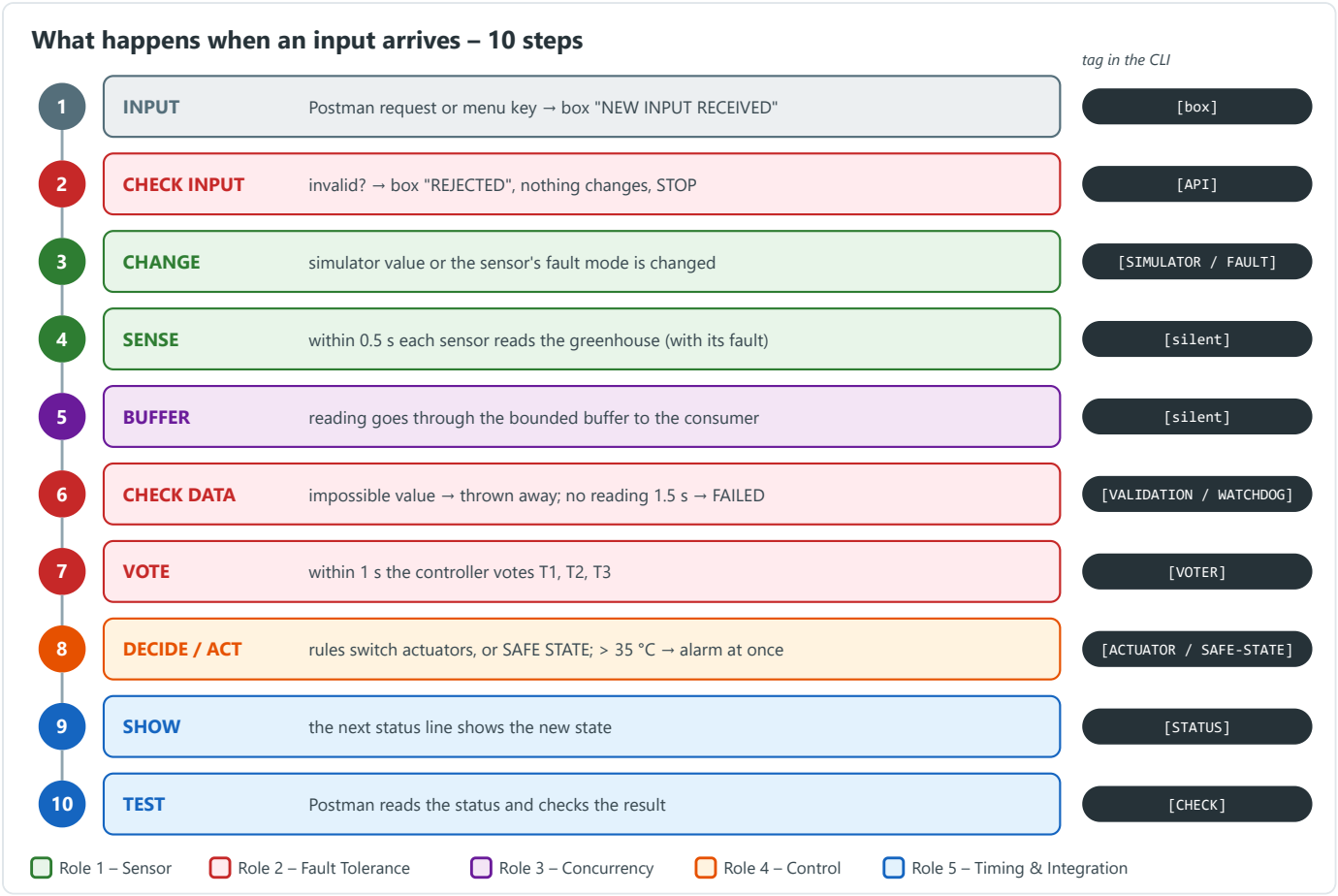
```
================================== NEW INPUT RECEIVED ==================================
  11:50:41.463  from Postman:  POST /api/environment
  RESULT        : REJECTED - 400 Validation failed [temperature must be <= 80 C (got 500.0)]
  SYSTEM REACTION: none - invalid input is refused, the greenhouse keeps running unchanged
========================================================================================
```

Scenario start (`#####` box) – a Postman folder begins (e.g. `F1-WrongValue`) and says what should happen. **Summary** (`-----` box) – at the end of the folder: numbered list of everything important that happened. **HOW TO READ THIS SCREEN** – printed at start-up; press `h` to show it again.

---

3. What happens when an input arrives – step by step

Every input goes through the same path:



Example A – F1: T2 gives a wrong value (real output)

```
NEW INPUT RECEIVED  POST /api/sensors/T2/fault {type=WRONG_VALUE, value=15.0}      ← 1. input arrives
11:50:04.222 [FAULT ] >>> FAULT injected: T2 -> WRONG_VALUE (+15.0 C)               ← 3. T2 is now broken
                                                                                     ← 4-6. T2 now sends ~40 C
(silent)
11:50:04.224 [VOTER ] >>> OK -> MASKED : T2=39.9 C out-voted, T1 and T3 agree      ← 7. voter ignores T2
                                    -> voted T=25.0 C
11:50:05.225 [STATUS] T=25.0C [MASKED 2/3] ... Heater off Fan off ... Alarm off | T2:WRONG_VALUE   ← 9. state
11:50:06.313 [CHECK ] Postman reads the status -> vote MASKED, alarm false          ← 10. Postman test passes
```

What it shows: T2 says 40 °C, but the system still uses the real 25 °C. The fan and alarm stay **off**. The fault (broken sensor) produced an error (wrong reading), but **no failure** (no wrong action).

Then the repair:

```
NEW INPUT RECEIVED  DELETE /api/sensors/T2/fault
11:50:06.408 [FAULT ] +++ T2 repaired - sensor works normally again
11:50:07.225 [VOTER ] +++ MASKED -> OK : all 3 agree (T1=25.2, T2=24.9, T3=25.1)
```

Example B – F2: T3 dead

```
NEW INPUT RECEIVED  POST /api/sensors/T3/fault {type=DEAD}
[FAULT   ] >>> FAULT injected: T3 -> DEAD (stops sending readings)
            ... T3 sends nothing; the watchdog counts the time since T3's last reading ...
[VOTER   ] >>> OK -> DEGRADED : only 2 sensors (T1=25.2, T2=25.1) - they agree
[WATCHDOG] !!! T3 FAILED - no heartbeat for 2001 ms (timeout 1500 ms)
[RT3     ] >>> [RT3 HARD] T3 detected dead 1611 ms after it stopped -> MET (deadline 2500 ms)
```

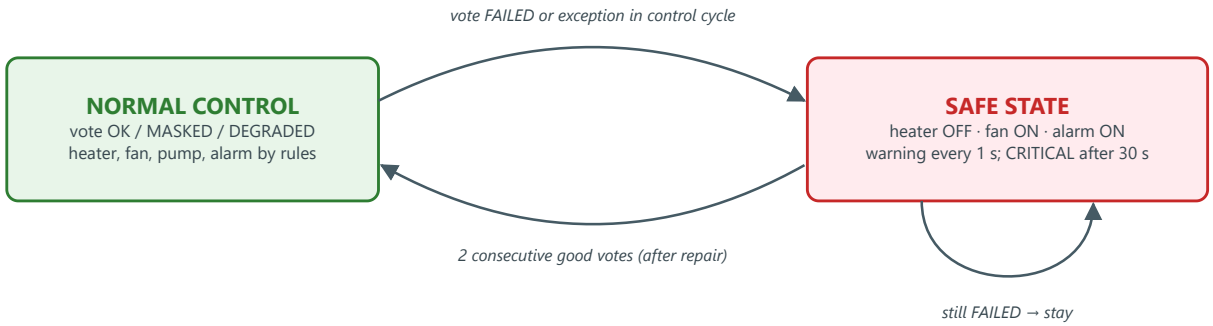
What it shows: a dead sensor is detected in 1.6 s (limit 2.5 s). The system keeps working with 2 sensors (DEGRADED).

Example C – F3: second fault → safe state

Smart Greenhouse Controller – CLI (F3: safe state)

```
================================== NEW INPUT RECEIVED ==================================
  11:50:22.458  from Postman:  POST /api/sensors/T1/fault  {type=WRONG_VALUE, value=15.0}
  WHAT IT MEANS : temperature sensor T1 will report a WRONG value (T3 is already dead)
----------------------------------------------------------------------------------------
11:50:22.459 WARN  [FAULT     ] >>> FAULT injected: T1 -> WRONG_VALUE (+15.0 C)
11:50:23.233 WARN  [VOTER     ] >>> DEGRADED -> FAILED : only 2 sensors and they DISAGREE (40.0 / 24.9)
11:50:23.233 ERROR [SAFE-STATE] !!! ENTER SAFE STATE - no trustworthy temperature
11:50:23.233 ERROR [SAFE-STATE] !!! Failure prevented: heater OFF, fan ON, alarm ON
11:50:23.234 INFO  [ACTUATOR  ] >>> FAN    -> ON  (safe state)
11:50:23.234 INFO  [ACTUATOR  ] >>> ALARM  -> ON  (safe state)
11:50:24.233 WARN  [ALARM     ] !!! SAFE STATE ACTIVE for 1 s - reason: no trustworthy temperature
                ... repair T1 and T3 ...
11:50:30.231 INFO  [SAFE-STATE] !!! Good vote 1/2 - confirming data before leaving safe state
11:50:31.232 INFO  [SAFE-STATE] +++ EXIT SAFE STATE after 2 consecutive good votes
11:50:31.232 INFO  [ACTUATOR  ] >>> ALARM  -> OFF (T=24.6 C <= 35.0 C)
```

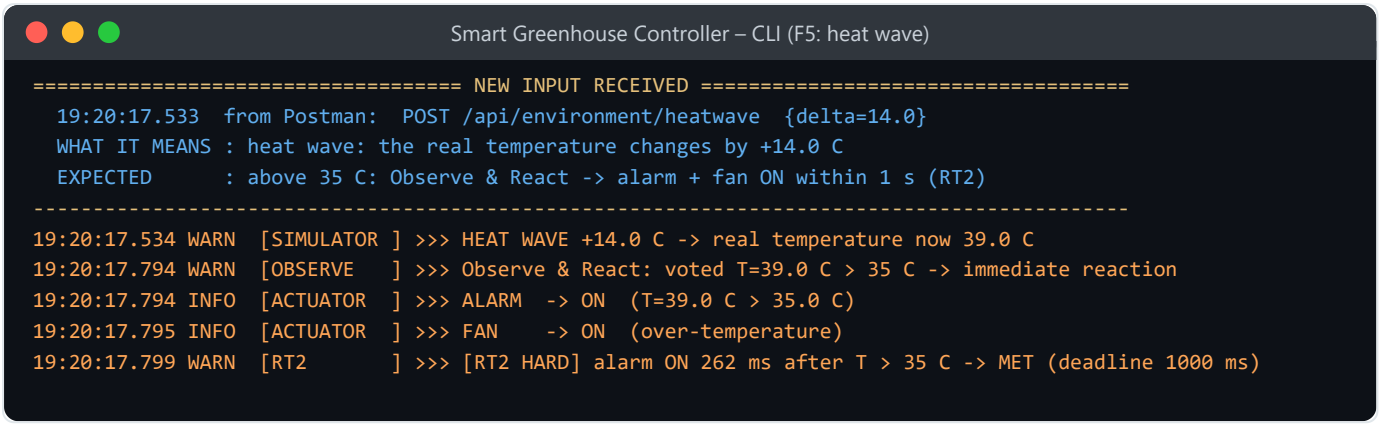
Safe state – fail-safe behaviour and forward recovery



Why these outputs? Without a trusted temperature the worst danger is overheating: cooling and warning a human is the safe side.
The system never stops itself – stopping would also switch the fan off.

What it shows: with 2 sensors that disagree, the system cannot know the real temperature, so it goes to the safe state (heater OFF, fan ON, alarm ON), warns every second, and leaves automatically after the repair.

Example D – F5: heat wave



What it shows: the alarm does not wait for the 1 s control cycle. It reacts on the first readings above 35 °C.

Example E – invalid input

```
NEW INPUT RECEIVED  POST /api/environment
RESULT        : REJECTED - 400 Validation failed [temperature must be <= 80 C (got 500.0)]
SYSTEM REACTION: none - invalid input is refused, the greenhouse keeps running unchanged
```

---

4. Small things that can look strange

You see	Why
`WAITING -> OK` after every reset	After a reset the voter starts fresh. "WAITING" means "no vote yet". One second later it votes OK.
The status line still shows the old state for a moment	The status line is printed every 2 s. It can be printed a few ms before the controller's change. The next status line is correct.
`SAFE STATE ACTIVE ...` many times	On purpose: the alarm is repeated every second while the problem lasts (the summary shows it once as `repeated xN`).
A short `MASKED` when the temperature is changed	The 3 sensors read a few ms apart. For one cycle one sensor may still show the old value. The next cycle is OK again.
`buf 3/10` in the status line	3 readings are in the buffer right now – the consumer empties it all the time, so this is normal.
Nothing new for a while	Nothing is happening – only the status line every 2 s. The system is waiting for the next input.

5. Tips

- **Read the box, then the lines under it.** The box tells you what arrived and what to expect. The lines under it are the system's real reaction.
- **For the presentation, use slow mode.** In Postman pick **Greenhouse-Presentation** (3 s between steps), or run `run-demo.bat slow`.
- **Read a scenario calmly afterwards.** Use the numbered **SUMMARY** at the end of each scenario, or open `logs/<date>/events_<time>.log`. That file contains only the boxes, events and summaries, with no status lines.
- **Don't click inside the black window.** Selecting text there pauses the output. Maximize the window so lines don't wrap.
- **Show the legend again** by pressing `h` in the CLI.